TINY LANGUAGE CFG RULES

Doaa Shaker Mohamed Aziz 24P0445
Malak Moustafa Ali Saleh 24P0239
Natalie David Nashaat Salib 24P0125
Nour Mostafa Abdelaty Melegy 24P0176

Final CFG:

Program → FunctionList Main_Function
FunctionList → Function_Statement FunctionList | ε
Main_Function → Datatype main ( ) Function_Body
Function_Statement → Function_Declaration Function_Body
Function_Declaration → Datatype FunctionName (ParameterList)
FunctionName → Identifier

Function_Body → { Statements Return_Statement }

Statements → Statement Statements | ε

Statement → Declaration_Statement

| Write_Statement

| Read_Statement

| If_Statement

| Repeat_Statement

| Comment

| Identifier_Statement

Declaration_Statement → Datatype IdList ;

Identifier_Statement → Identifier Identifier_Tail

Identifier_Tail → := Expression ; | (Arguments) ;

Assignment_Statement → Identifier := Expression ;

Write_Statement → write Write_Tail ;

Write_Tail → endl | Expression

Read_Statement → read Identifier ;

Return_Statement → return Expression ;

Function_Call_Statement → Function_Call ;

IdList → Identifier IdList_Init IdList'

IdList_Init → := Expression | ε

IdList' → , Identifier IdList_Init IdList' | ε

ParameterList → Parameter MoreParameters | ε

MoreParameters → , Parameter MoreParameters | ε

Parameter → Datatype Identifier

Arguments → Term MoreArgs | ε

MoreArgs → , Term MoreArgs | ε

If_Statement → if Condition_Statement then Statements If_Tail

If_Tail → elseif Condition_Statement then Statements If_Tail | Else_Part | end

Else_Part → else Statements end

Repeat_Statement → repeat Statements until Condition_Statement

Condition_Statement → Condition Condition_Ext

Condition_Ext → Boolean_Operator Condition Condition_Ext | ε

Condition → Identifier Condition_Operator Term

Condition_Operator → < | > | = | <>

Boolean_Operator → && | ||

Expression → StringLiteral | Equation

Equation → Factor Equation_Tail

Equation_Tail → AddOp Factor Equation_Tail | ε

AddOp → + | -

Factor → Term Factor_Tail

Factor_Tail → MultOp Term Factor_Tail | ε

MultOp → * | /

Term → Number | Function_Call | (Equation)

Function_Call → Identifier (Arguments)

Datatype → int | float | string

Comment → /* Comment_Content */

Comment_Content → Comment_Char Comment_Content | ε

Left Recursion and Left Factoring Elimination:

1. Equation — Left Recursion Elimination

Before:

Equation → Equation AddOp Factor | Factor

Step:

A → A α | β becomes

A → β A'

A' → α A' | ε

β = Factor

α = AddOp Factor

After:

Equation → Factor Equation_Tail

Equation_Tail → AddOp Factor Equation_Tail | ε

2. Factor — Left Recursion Elimination

Before:

Factor → Factor MultOp Term | Term

Step:

A → A α | β becomes

A → β A'

A' → α A' | ε

β = Term

α = MultOp Term

After:

Factor → Term Factor_Tail

Factor_Tail → MultOp Term Factor_Tail | ε

3. IdList — Left Recursion Elimination

Before:

IdList → IdList , Identifier IdList_Init | Identifier IdList_Init

Step:

A → A α | β becomes

A → β A'

A' → α A' | ε

β = Identifier IdList_Init

α = , Identifier IdList_Init

After:

IdList → Identifier IdList_Init IdList'

IdList' → , Identifier IdList_Init IdList' | ε

4. MoreParameters — Left Recursion Elimination

Before:

MoreParameters → MoreParameters , Parameter | Parameter | ε

Step:

A → A α | β becomes

A → β A'

A' → α A' | ε

β = Parameter

α = , Parameter

After:

ParameterList → Parameter MoreParameters | ε

MoreParameters → , Parameter MoreParameters | ε

5. Arguments — Left Recursion Elimination

Before:

Arguments → Arguments , Term | Term | ε

Step:

A → A α | β becomes

A → β A'

A' → α A' | ε

β = Term

α = , Term

After:

Arguments → Term MoreArgs | ε

MoreArgs → , Term MoreArgs | ε

6. Statement — Left Factoring

Before:

Statement → Identifier := Expression ;

| Identifier (Arguments) ;

Step:

A → α β | α γ becomes

A → α A'

A' → β | γ

Common prefix:

α = Identifier

After:

Statement → Identifier_Statement | ...

Identifier_Statement → Identifier Identifier_Tail

Identifier_Tail → := Expression ;

| (Arguments) ;

7. Function_Declaration vs Main_Function — Left Factoring

Before:

Function_Statement → Datatype Identifier (ParameterList) Function_Body

Main_Function → Datatype main ( ) Function_Body

Step:

A → α β | α γ becomes

A → α A'

A' → β | γ

Common prefix:

α = Datatype

After:

Function_Declaration → Datatype FunctionName (ParameterList)

FunctionName → Identifier

Main_Function → Datatype main ( ) Function_Body

Resolution:

The scanner emits "main" as a separate token (Main),

not as an Identifier.

After reading Datatype:

- if next token is Main → parse Main_Function

- if next token is Identifier → parse Function_Statement